STUDY 2

Anomaly Detection

Automatically identifying data points that are surprisingly different from normal

Study Discussion

What Is Anomaly Detection?

Anomaly detection means automatically finding data points that are surprisingly different from what is normal. Like a doctor looking at a heart rate monitor: 72, 74, 71, 73, 72, 74, 180... something is wrong!

Real World Examples: Fraud detection (credit card used in 3 countries in 1 hour), server monitoring (CPU spikes to 99% at 3am), manufacturing (machine vibrating differently before it breaks), finance (stock drops 40% in 5 minutes), e-commerce (user adds 500 items to cart, suggesting bot behavior).

Step 0: Clarify the Problem First

- **What kind of data?** Time series (CPU over time)? Tabular (one row per transaction)? Or a single snapshot?
- **Labeled or unlabeled?** Do we have examples of past anomalies? Most real-world anomaly problems are unlabeled.
- **How fast do we need to detect it?** Real-time (fraud while card is swiped)? Or batch (daily report)?
- **Cost of false alarm vs. missed anomaly?** Missing fraud = lose money. False alarm on fraud = block a legitimate customer.
- **How rare are anomalies?** Usually < 1% of data, which makes this a class imbalance problem.

Step 1: Types of Anomalies

Not all anomalies are the same. There are 3 distinct types you must know:

Type 1: Point Anomaly

A single data point is weird. Example: CPU = 45%, 47%, 46%, 45%, 98%, 46%, 45%; one bad point stands out.

Type 2: Contextual Anomaly

A value that is normal in one context but weird in another. CPU at 90% at 2pm on a weekday = NORMAL (peak traffic). CPU at 90% at 3am on Sunday = ANOMALY. Same value, completely different meaning depending on context. This is why time-of-day features matter so much.

Type 3: Collective Anomaly

No single point is weird, but a pattern is weird. CPU stuck at exactly 65% for 6 hours; each individual value is normal, but the pattern of being perfectly static for that long is suspicious (maybe a runaway process).

Step 2: Exploring the Data

Statistical Description of Normal

Before building any model, understand what normal looks like statistically. You need the mean and standard deviation of the normal behavior.

The Empirical Rule (68-95-99.7 Rule)

- 68% of data falls within μ ± 1σ
- 95% of data falls within μ ± 2σ
- 99.7% of data falls within μ ± 3σ

Anything beyond 3σ from the mean is very unusual; only 0.3% of normal data lives there. Example: CPU mean = 45%, σ = 10%. Normal range = [15%, 75%]. If CPU = 98% → it is 5.3σ away → ANOMALY.

Step 3: Feature Engineering

For Time Series Metrics (CPU, Latency, etc.)

- `rolling_mean_5min`: average over last 5 minutes
- `rolling_std_5min`: how much it fluctuated
- `delta` = current_value - value_5min_ago (rate of change)
- `z_score` = (current_value - rolling_mean) / rolling_std
- `ratio` = current_latency / avg_latency_same_hour_yesterday

The Z-Score Feature

The z-score answers: how many standard deviations away from recent normal is this point? A z-score of 1 = slightly unusual. A z-score of 4 = very suspicious. This is the single most important feature for anomaly detection.

For Tabular Data (Transactions, User Behavior)

- `amount_vs_user_avg`: how unusual is this amount for THIS user?
- `time_since_last_transaction`: 3 transactions in 10 seconds = suspicious
- `transactions_last_hour`: velocity check
- `is_new_country`: user never bought from here before
- `is_new_device`: new device ID

Step 4: Statistics and Math

Z-Score

z = (x - μ) / σ. Flag as anomaly if |z| > 3. Simple and fast. Works well for normally distributed data. Problem: real data is often skewed.

Modified Z-Score (More Robust)

Instead of mean and std, use median and MAD (Median Absolute Deviation). The median is resistant to outliers, so one extreme value does not drag it.

- $MAD = median(|x_i - median(x)|)$
- $Modified\ Z = 0.6745 \times (x - median) / MAD$

IQR Method (Interquartile Range)

- $Q1 = 25th\ percentile,\ \ Q3 = 75th\ percentile$
- $IQR = Q3 - Q1$
- $Lower\ bound = Q1 - 1.5 \times IQR$
- $Upper\ bound = Q3 + 1.5 \times IQR$

Anything outside these bounds is an outlier. No assumption about distribution shape. This is what box plots show.

Mahalanobis Distance (For Multiple Features)

What if you have 2 metrics, CPU and memory? A point might look normal on each individually but anomalous in combination. CPU = 70% and Memory = 95% together is unusual even if each alone might be seen sometimes.

Mahalanobis Distance measures how far a point is from the center of the distribution, accounting for the correlation between features:

- $D^2 = (x - \mu)^T \Sigma^{-1} (x - \mu)$
- x = your data point (vector of all features)
- μ = mean vector

- Σ^{-1} = inverse of the covariance matrix (captures how features relate to each other)

This is a generalization of z-score to multiple dimensions. It accounts for correlations between features when measuring distance.

Step 5: Models: From Simple to Complex

Category 1: Statistical Methods

3-Sigma Rule

Flag if: |x - mean| > 3 × std. Instant, no training needed. Assumes normal distribution, sensitive to outliers in the baseline period.

CUSUM (Cumulative Sum)

Designed to detect when a metric shifts to a new level, not just spikes. S_t = max(0, S_{t-1} + (x_t - μ - k)). When S_t exceeds a threshold → alert. Analogy: you are weighing yourself daily. CUSUM detects when your weight has been consistently trending upward for a week, not just one heavy meal.

- **Pros:** Great for detecting sustained shifts, widely used in manufacturing quality control

Category 2: Unsupervised ML (No Labels Needed)

These are the most commonly used because you almost never have labeled anomaly data in practice.

Isolation Forest

Intuition: anomalies are easy to isolate. Normal points are surrounded by neighbors, so you need many random splits to isolate them. Anomalies are alone, so you isolate them in very few splits. The anomaly score is 1 / (average path length). Higher score = more anomalous.

- **Pros:** Works well on high-dimensional data, fast, no assumptions about distribution
- **Cons:** Does not capture time patterns (each point treated independently)

Local Outlier Factor (LOF)

A point is anomalous if its neighborhood is much denser than it is. LOF score ≈ 1 means normal density; LOF >> 1 means the point is in a sparser region than its neighbors. Analogy: people clustered at a party; the anomaly is someone standing alone far from all groups.

- **Pros:** Detects local anomalies even when different clusters have different densities
- **Cons:** Slow for large datasets because computing k-nearest neighbors is expensive

Autoencoders: The Most Powerful Unsupervised Method

Train a neural network to compress data and then reconstruct it. Trained only on normal data, it gets good at reconstructing normal patterns. When it sees an anomaly, it cannot reconstruct it well → high reconstruction error = anomaly.

- Input: [CPU=45%, mem=60%, latency=200ms]
- Encoder compresses to small bottleneck vector: [0.3, -0.7]
- Decoder reconstructs: [CPU=44%, mem=61%, latency=201ms]
- Reconstruction error = |Input - Output| → small for normal, large for anomaly
- **Pros:** Can handle complex multivariate patterns, works for tabular, time series, and image data
- **Cons:** Needs enough normal data to train on, hard to interpret why something is anomalous

Variant: LSTM Autoencoder uses LSTM layers to capture temporal patterns in sequences. Great for time series anomaly detection.

One-Class SVM

Learns a boundary around all normal data. Anything outside the boundary = anomaly. SVM finds a hyperplane that separates normal data from the origin in a transformed feature space.

- **Pros:** Works well with smaller datasets
- **Cons:** Does not scale well to large datasets, sensitive to hyperparameter choices

Category 3: Supervised ML (When You Have Labels)

If you DO have labeled examples of past anomalies (rare but valuable), use XGBoost or LightGBM as a classifier.

Handling Class Imbalance

If 99% of transactions are normal and 1% are fraud, a model that always predicts normal gets 99% accuracy but catches zero fraud.

- **SMOTE:** Create synthetic anomaly examples by interpolating between existing anomalies
- **Class weights:** Tell the model that missing a fraud is 100x worse than a false alarm
- **Better metrics:** Precision, Recall, F1, AUC-PR (not accuracy!)

Category 4: Time-Series Specific Models

STL + Residual Analysis

Decompose the series: y(t) = Trend + Seasonality + Residual. After removing normal patterns (weekly cycle, growth trend), the residuals should be random noise. A large residual = something unusual happened. Flag residuals beyond 3σ.

ARIMA-Based Anomaly Detection

Fit ARIMA on historical normal data. At each new time point, ARIMA predicts the expected value and a confidence interval. If actual value falls outside the interval → anomaly. ARIMA predicts: 45% CPU ± 10%. Actual: 92% → outside [35%, 55%] → ANOMALY.

Facebook Prophet for Anomaly Detection

Fit Prophet model, generate prediction intervals (e.g., 99% confidence), and flag points outside them. Very practical for business metric monitoring with daily/weekly patterns.

Model Selection Guide

Scenario	Recommended Model
Have labeled anomalies	XGBoost/LightGBM with class weights + SMOTE
Time series, need baseline	STL residuals + 3-sigma rule
Time series, need intervals	ARIMA/Prophet confidence intervals
Time series, best accuracy	LSTM Autoencoder
Tabular, no labels	Isolation Forest
Multi-feature relationships matter	Autoencoder (captures correlations)
Small dataset, no labels	One-Class SVM

Step 6: Evaluation

The Label Problem

- **If you have labels:** Use Precision, Recall, F1, AUC-PR (NOT accuracy). Use time-based train/test split.
- **Inject synthetic anomalies:** Take normal data, artificially inject anomalies, evaluate how many you catch.
- **Human evaluation:** Deploy model, send alerts to humans, track what % were actually real anomalies.

The Threshold Problem

Most anomaly models give a score, not a binary yes/no. You choose a threshold. Lower threshold = more alerts = higher recall, lower precision. Higher threshold = fewer alerts = lower recall, higher precision.

Practical advice: Start conservative (high threshold → fewer alerts). Nothing kills trust in a system faster than too many false alarms. Engineers start ignoring alerts → you miss real ones.

Step 7: Production

Real-Time vs. Batch Detection

- **Real-time (online):** Score each point as it arrives. Use for fraud detection, server monitoring. Model must be very fast (milliseconds). Use stateless models: Z-score, Isolation Forest.
- **Batch:** Score a window of data periodically. Use for daily business metric reviews, log analysis. Can use heavier models: LSTM Autoencoder.

Handling Seasonality in Production

Your server always has high CPU from 9am–5pm. Your model should not alert on that. Fix: train separate models for different contexts (business hours vs. off-hours, weekday vs. weekend). Or condition on time as a feature so the model learns that 80% CPU is normal at 2pm.

Concept Drift

The definition of normal changes over time: your app grows, baseline CPU increases; you deploy a new service, memory usage pattern changes; holiday season brings different traffic patterns. Fix: Use a rolling training window and train only on the last 30 days of data, not all history.

Alert Fatigue: The Real Enemy

The #1 failure mode in production anomaly detection is NOT a bad model; it is too many alerts. If engineers get 50 alerts per day and most are false alarms, they start ignoring all alerts, and a real critical issue gets missed.

- **Alert grouping:** Multiple anomalies from the same root cause → one alert
- **Severity levels:** P1 (wake me up now) vs P3 (email in morning)
- **Anomaly correlation:** CPU + memory + latency all spike together → probably one real issue, not 3 alerts
- **Feedback loops:** Track alert resolution → use to tune threshold over time

How to Discuss the Problem

First clarify: is this labeled or unlabeled data? Real-time or batch? What is the cost of false positives vs. false negatives? For features, engineer rolling statistics such as z-scores, rate of change. For time series, decompose using STL and look at residuals. Context features like hour of day matter a lot. Start with Isolation Forest as baseline. If sequence data, use LSTM Autoencoder. If labels exist, use LightGBM with class weights and evaluate on precision-recall. In production, the real challenge is alert fatigue. Start with a conservative threshold, build a human feedback loop, and use that to tune the model over time.